

Построение жордановой нормальной формы в точной символьной арифметике

Artur

6 мая 2026 г.

Содержание

1	Постановка задачи	3
2	Используемые технологии	3
2.1	Запрещённые функции	4
3	Теоретические сведения	4
3.1	Жорданова нормальная форма	4
3.2	Обобщённые собственные векторы	4
3.3	Размеры жордановых клеток	5
4	Описание алгоритма	5
4.1	Вычисление характеристического многочлена	5
4.2	Поиск собственных значений	5
4.3	Построение корневых подпространств	5
4.4	Построение жордановых цепочек	6
5	Описание реализованных функций	6
5.1	Функции линейной алгебры	6
5.1.1	<code>matrix_rank(A)</code>	6
5.1.2	<code>defect(A)</code>	6
5.1.3	<code>nullspace(A)</code>	6
5.1.4	<code>is_independent(vectors)</code>	6
5.1.5	<code>independent_after_adding(current_basis, new_vectors)</code>	7
5.2	Функции работы с характеристическим многочленом	7
5.2.1	<code>characteristic_polynomial(A)</code>	7
5.2.2	<code>algebraic_roots_with_multiplicities(A)</code>	7
5.3	Функции построения ЖНФ	7
5.3.1	<code>shifted_matrix(A, lam)</code>	7
5.3.2	<code>jordan_block_sizes(A, lam, algebraic_multiplicity)</code>	7
5.3.3	<code>build_chain_from_top(N, top, size)</code>	7
5.3.4	<code>construct_chains_for_eigenvalue(A, lam, block_sizes)</code>	7
5.3.5	<code>jordan_form_exact(A)</code>	8
5.4	Функции генерации тестов	8
5.4.1	<code>jordan_block(lam, size)</code>	8

5.4.2	<code>make_jordan_matrix(block_specs)</code>	8
5.4.3	<code>random_unimodular_matrix(n)</code>	8
5.4.4	<code>same_jordan_structure(structure₁, structure₂)</code>	8
5.4.5	<code>generated_test(block_specs)</code>	9
5.5	Функции демонстрации и проверки	9
5.5.1	<code>demo_manual(A, name)</code>	9
6	Генератор тестов	9
6.1	Построение тестовой матрицы	9
6.2	Почему используются унимодулярные матрицы	10
7	Проверка корректности	10
7.1	Проверка подобия	10
7.2	Проверка невырожденности	10
7.3	Проверка структуры блоков	10
8	Набор тестов	10
9	Пример	11
10	Структура проекта	11
11	Инструкция по запуску	11
12	Вывод	11

1 Постановка задачи

Цель работы — реализовать алгоритм построения жордановой нормальной формы (ЖНФ) матрицы в точной арифметике без использования встроенных функций вычисления ЖНФ, собственных значений и собственных векторов.

Пусть дана квадратная матрица

$$A \in M_n(F),$$

где элементы матрицы принадлежат полю рациональных чисел или их алгебраическому расширению.

Требуется:

- вычислить точные собственные значения;
- определить размеры жордановых клеток;
- построить базис из обобщённых собственных векторов;
- построить матрицу перехода P ;
- построить жорданову матрицу J , такую что

$$A = PJP^{-1}.$$

Также необходимо реализовать автоматический генератор тестов и систему автоматической проверки корректности алгоритма.

2 Используемые технологии

Для реализации использовался язык программирования Python и библиотека SymPy.

Используемые возможности SymPy:

- хранение матриц;
- символьная арифметика;
- вычисление характеристического многочлена;
- факторизация многочленов;
- вычисление ядра матрицы;
- вычисление ранга;
- обращение матриц.

2.1 Запрещённые функции

В работе не использовались:

- `Matrix.jordan_form()`;
- `eigenvals()`;
- `eigenvects()`;
- встроенные реализации SVD.

Построение жордановых цепочек и логика алгоритма реализованы вручную.

3 Теоретические сведения

3.1 Жорданова нормальная форма

Для любой матрицы A над алгебраически замкнутым полем существует невырожденная матрица P , такая что

$$A = PJP^{-1},$$

где J — жорданова матрица.

Жорданова клетка размера k для собственного значения λ :

$$J_k(\lambda) = \begin{pmatrix} \lambda & 1 & 0 & \cdots & 0 \\ 0 & \lambda & 1 & \cdots & 0 \\ 0 & 0 & \lambda & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & \lambda \end{pmatrix}.$$

3.2 Обобщённые собственные векторы

Пусть

$$N = A - \lambda I.$$

Вектор v называется обобщённым собственным вектором высоты k , если

$$N^k v = 0,$$

но

$$N^{k-1} v \neq 0.$$

Жорданова цепочка имеет вид

$$v_1, \dots, v_k,$$

где

$$Nv_{i+1} = v_i.$$

3.3 Размеры жордановых клеток

Обозначим

$$d_k = \dim \ker(A - \lambda I)^k.$$

Тогда количество блоков размера не меньше k :

$$b_k = d_k - d_{k-1}.$$

Количество блоков размера ровно k :

$$e_k = b_k - b_{k+1}.$$

Эти формулы использовались для определения структуры ЖНФ.

4 Описание алгоритма

4.1 Вычисление характеристического многочлена

Для матрицы A вычисляется характеристический многочлен

$$\chi_A(x) = \det(xI - A).$$

После этого выполняется факторизация многочлена.

4.2 Поиск собственных значений

Из факторизации характеристического многочлена извлекаются точные алгебраические корни.

Например:

$$x^2 - 2 = 0$$

даёт собственные значения

$$\lambda_1 = \sqrt{2}, \quad \lambda_2 = -\sqrt{2}.$$

4.3 Построение корневых подпространств

Для каждого собственного значения вычисляются ядра матриц

$$(A - \lambda I)^k.$$

Используются встроенные символьные методы поиска ядра, однако логика построения ЖНФ реализуется вручную.

4.4 Построение жордановых цепочек

Пусть

$$N = A - \lambda I.$$

Если найден вектор w , удовлетворяющий

$$N^s w = 0,$$

то строится цепочка

$$N^{s-1}w, \dots, Nw, w.$$

Столбцы всех цепочек образуют матрицу перехода P .

5 Описание реализованных функций

В программе были реализованы следующие функции.

5.1 Функции линейной алгебры

5.1.1 `matrix_rank(A)`

Вычисляет ранг матрицы (A).

Используется при:

- вычислении дефектов;
- проверке линейной независимости;
- определении размерности ядра.

5.1.2 `defect(A)`

Вычисляет дефект матрицы:

$$[\text{def}(A) = n - \text{rank}(A).]$$

Используется при вычислении размеров жордановых клеток.

5.1.3 `nullspace(A)`

Возвращает базис ядра матрицы:

$$[\ker A.]$$

Используется для построения обобщённых собственных векторов.

5.1.4 `is_independent(vectors)`

Проверяет линейную независимость набора векторов.

Используется при построении жордановых цепочек.

5.1.5 independent_after_adding(current_{basis}, new_{vectors})

Проверяет, остаётся ли система векторов линейно независимой после добавления новых векторов.

Используется при выборе корректных жордановых цепочек.

5.2 Функции работы с характеристическим многочленом

5.2.1 characteristic_polynomial(A)

Вычисляет характеристический многочлен матрицы:

$$[\chi_A(x) = \det(xI - A).]$$

5.2.2 algebraic_roots_with_multiplicities(A)

Находит точные собственные значения матрицы и их алгебраические кратности.

Поддерживаются:

- рациональные корни;
- иррациональные алгебраические числа;
- корни вида CRootOf.

5.3 Функции построения ЖНФ

5.3.1 shifted_matrix(A, lam)

Строит матрицу

$$[A - \lambda I.]$$

Используется при вычислении корневых подпространств.

5.3.2 jordan_block_sizes(A, lam, algebraic_{multiplicity})

Вычисляет размеры жордановых клеток для собственного значения (λ).

Используются формулы:

$$[d_k = \dim \ker(A - \lambda I)^k,]$$

$$[b_k = d_k - d_{k-1},]$$

$$[e_k = b_k - b_{k+1}.]$$

5.3.3 build_chain_from_top(N, top, size)

Строит жорданову цепочку из обобщённого собственного вектора (top).

Для

$$[N = A - \lambda I]$$

строится последовательность:

$$[N^{s-1}top, \dots, Ntop, top.]$$

5.3.4 construct_chains_for_eigenvalue(A, lam, block_{sizes})

Строит все жордановы цепочки для фиксированного собственного значения.

Обеспечивает линейную независимость цепочек.

5.3.5 `jordan_formexact(A)`

Основная функция программы.

Выполняет:

- вычисление собственных значений;
- вычисление размеров клеток;
- построение жордановых цепочек;
- построение матрицы перехода (P);
- построение жордановой матрицы (J).

Возвращает: [(J,P,structure).]

5.4 Функции генерации тестов

5.4.1 `jordan_block(lam, size)`

Создаёт одну жорданову клетку размера (size) для собственного значения (λ).

5.4.2 `make_jordan_matrix(blockspecs)`

Создаёт жорданову матрицу из набора жордановых клеток.

Пример: [[(3,2),(3,1),(-1,1)].]

5.4.3 `random_unimodular_matrix(n)`

Генерирует случайную унимодулярную матрицу размера ($n \times n$).

Используются элементарные преобразования строк:

- перестановка строк;
- прибавление кратной строки;
- умножение строки на (-1).

Для такой матрицы:

[$\det P = \pm 1$.]

5.4.4 `same_jordan_structure(structure1, structure2)`

Сравнивает две жордановы структуры:

- собственные значения;
- размеры клеток.

Используется в автоматических тестах.

5.4.5 `generated_test(blockspecs)`

Полностью автоматический тест:

1. Генерируется (J_0);
1. Генерируется унимодулярная матрица (P_0);
1. Вычисляется [$A=P_0J_0P_0^{-1}$;]
1. Матрица (A) подаётся на вход алгоритму;
2. Проверяется корректность восстановленной ЖНФ.

5.5 Функции демонстрации и проверки

5.5.1 `demo_manual(A, name)`

Запускает демонстрацию работы алгоритма на матрице (A).

Выводит:

- исходную матрицу;
- найденную ЖНФ;
- матрицу перехода;
- проверку условия [$AP=PJ$.]

6 Генератор тестов

Для проверки алгоритма реализован автоматический генератор тестов.

6.1 Построение тестовой матрицы

Сначала генерируется жорданова матрица заданной структуры:

$$J_0.$$

Затем генерируется случайная унимодулярная матрица

$$P_0,$$

такая что

$$\det P_0 = \pm 1.$$

После этого вычисляется тестовая матрица:

$$A = P_0J_0P_0^{-1}.$$

6.2 Почему используются унимодулярные матрицы

Если P_0 — унимодулярная матрица с целыми коэффициентами, то матрица

$$A = P_0 J_0 P_0^{-1}$$

сохраняет точность символьных вычислений и имеет хорошие арифметические свойства.

7 Проверка корректности

Корректность алгоритма проверяется следующими условиями.

7.1 Проверка подобия

Проверяется равенство

$$AP = PJ.$$

Эквивалентно:

$$A = PJP^{-1}.$$

7.2 Проверка невырожденности

Проверяется условие

$$\det P \neq 0.$$

7.3 Проверка структуры блоков

Сравниваются:

- размеры жордановых клеток;
- соответствующие собственные значения.

8 Набор тестов

Были реализованы следующие тесты:

- диагонализуемые матрицы;
- нильпотентные матрицы;
- одна максимальная жорданова клетка;
- несколько блоков одного собственного значения;
- иррациональные алгебраические собственные значения;
- смешанный спектр;
- автоматически сгенерированные тесты.

9 Пример

Рассмотрим матрицу

$$A = \begin{pmatrix} 0 & 2 \\ 1 & 0 \end{pmatrix}.$$

Характеристический многочлен:

$$\chi_A(x) = x^2 - 2.$$

Собственные значения:

$$\lambda_1 = \sqrt{2}, \quad \lambda_2 = -\sqrt{2}.$$

Матрица диагонализуема, поэтому

$$J = \begin{pmatrix} \sqrt{2} & 0 \\ 0 & -\sqrt{2} \end{pmatrix}.$$

10 Структура проекта

```
1 project/  
2 |  
3 |-- jordan_form_sympy_exact.py  
4 |-- test_jordan_form_sympy_exact.py  
5 |-- README.md
```

11 Инструкция по запуску

Установка зависимостей:

```
1 pip install sympy pytest
```

Запуск основной программы:

```
1 python jordan_form_sympy_exact.py
```

Запуск тестов:

```
1 pytest -q
```

12 Вывод

В ходе работы был реализован алгоритм построения жордановой нормальной формы в точной символьной арифметике.

Особенности реализации:

- поддержка алгебраических чисел;
- отсутствие численных ошибок;
- ручное построение жордановых цепочек;

- автоматическая генерация тестов;
- автоматическая проверка корректности.

Алгоритм успешно проходит все разработанные тесты и корректно восстанавливает структуру жордановой нормальной формы.